

Dokumentasi Project dbt: E-Commerce Analytics Engineering Pipeline

1. Gambaran Umum Project

Project ini dibuat sebagai simulasi workflow **Analytics Engineering** menggunakan **dbt Core** dan **PostgreSQL**.

Tujuan utamanya adalah membangun alur data dari data mentah hingga menjadi tabel final yang siap dipakai untuk analisis dan dashboard.

Alur besar project:

```
graph TD; A[Data dummy CSV] --> B[dbt seed]; B --> C[PostgreSQL raw tables]; C --> D[dbt staging models]; D --> E[dbt intermediate models]; E --> F[dbt marts models]; F --> G[dbt test]; G --> H[dbt docs]; H --> I[Power BI dashboard];
```

Project ini mensimulasikan kondisi dunia kerja, di mana data dari sistem bisnis seperti aplikasi e-commerce, sistem pembayaran, dan data customer perlu dibersihkan, digabungkan, diuji, dan disiapkan sebelum digunakan oleh dashboard.

Dalam dunia nyata, alurnya biasanya:

```
graph TD; A[Aplikasi bisnis] --> B[Data ingestion / pipeline]; B --> C[Data warehouse]; C --> D[dbt transformation]; D --> E[ ];
```

```
Data mart
↓
BI dashboard
```

Dalam project ini, karena belum ada aplikasi sungguhan, maka data dummy CSV digunakan sebagai pengganti data mentah dari sistem bisnis.

2. Tools yang Digunakan

Tools	Fungsi
PostgreSQL	Database untuk menyimpan raw data dan hasil transformasi
dbt Core	Tools utama untuk transformasi data berbasis SQL
dbt-postgres	Adapter agar dbt bisa terhubung ke PostgreSQL
VS Code	Editor untuk menulis file SQL, YAML, dan konfigurasi
pgAdmin	Mengecek tabel, view, dan schema di PostgreSQL
Power BI	Tahap berikutnya, untuk membuat dashboard
GitHub	Tahap berikutnya, untuk menyimpan portfolio

Stack project:

```
dbt Core + PostgreSQL + SQL + Power BI + GitHub
```

3. Setup Awal dbt dan PostgreSQL

3.1 Membuat Database PostgreSQL

Kita membuat database khusus untuk project ini:

```
create database ecommerce_analytics;
```

Tujuan database ini adalah menjadi tempat penyimpanan data project e-commerce, baik data mentah maupun data hasil transformasi.

Database yang digunakan:

```
ecommerce_analytics
```

3.2 Membuat Schema Awal

Kita juga menyiapkan konsep schema untuk memisahkan layer data:

```
raw
staging
intermediate
marts
```

Namun pada praktik awal, data seed masuk dulu ke schema `public`.

Setelah project dirapikan, output dbt dipisahkan menjadi:

```
public
public_staging
public_intermediate
public_marts
```

Penjelasan:

Schema	Fungsi
<code>public</code>	Menyimpan raw data dari CSV seed
<code>public_staging</code>	Menyimpan hasil staging model
<code>public_intermediate</code>	Menyimpan hasil intermediate model
<code>public_marts</code>	Menyimpan tabel final untuk analisis dan dashboard

4. Install dbt untuk PostgreSQL

Karena database yang digunakan adalah PostgreSQL, maka dbt membutuhkan adapter `dbt-postgres`.

Command yang digunakan:

```
python -m pip install dbt-postgres
```

Tujuannya agar dbt dapat terhubung dan menjalankan SQL di PostgreSQL.

Setelah install, kita cek versi dbt:

```
dbt --version
```

Tujuan command ini adalah memastikan bahwa dbt Core dan adapter PostgreSQL sudah berhasil terinstall.

5. Membuat Project dbt

Project dibuat dengan command:

```
dbt init dbt_ecommerce
```

Nama project:

```
dbt_ecommerce
```

Struktur awal project kurang lebih seperti ini:

```
dbt_ecommerce/  
├─ models/  
├─ seeds/  
├─ dbt_project.yml  
└─ ...
```

Penjelasan:

Folder/File	Fungsi
<code>models/</code>	Tempat menyimpan file SQL transformasi
<code>seeds/</code>	Tempat menyimpan CSV yang akan dimasukkan ke database
<code>dbt_project.yml</code>	Konfigurasi utama project dbt
<code>profiles.yml</code>	Konfigurasi koneksi dbt ke database

6. Konfigurasi Koneksi dbt ke PostgreSQL

Agar dbt bisa terhubung ke PostgreSQL, kita mengatur file:

```
C:\Users\NAMA_USER\.dbt\profiles.yml
```

Isi konfigurasi:

```
dbt_ecommerce:  
  target: dev
```

```

outputs:
  dev:
    type: postgres
    host: localhost
    user: postgres
    password: password_postgresql_anda
    port: 5432
    dbname: ecommerce_analytics
    schema: public
    threads: 4

```

Penjelasan:

Konfigurasi	Fungsi
<code>type: postgres</code>	Memberi tahu dbt bahwa database yang digunakan adalah PostgreSQL
<code>host: localhost</code>	PostgreSQL berjalan di komputer lokal
<code>user: postgres</code>	Username database
<code>password</code>	Password PostgreSQL
<code>port: 5432</code>	Port default PostgreSQL
<code>dbname: ecommerce_analytics</code>	Database yang digunakan
<code>schema: public</code>	Schema default output dbt
<code>threads: 4</code>	Jumlah proses paralel yang bisa digunakan dbt

Di file `dbt_project.yml`, nama profile harus sama:

```
profile: 'dbt_ecommerce'
```

Hubungannya:

```

dbt_project.yml
↓
profile: dbt_ecommerce
↓
profiles.yml
↓
PostgreSQL ecommerce_analytics

```

7. Mengecek Koneksi dengan `dbt debug`

Command:

```
dbt debug
```

Tujuan:

Mengecek apakah dbt sudah bisa membaca project, membaca profiles.yml, dan terhubung ke PostgreSQL.

Jika berhasil, output menunjukkan:

```
All checks passed!
```

Artinya:

```
dbt berhasil membaca konfigurasi project
dbt berhasil membaca koneksi database
dbt berhasil terhubung ke PostgreSQL
database ecommerce_analytics bisa diakses
```

Command `dbt debug` digunakan saat:

```
baru setup project
mengubah profiles.yml
mengganti password database
mengalami error koneksi
```

8. Membuat Data Dummy Menggunakan CSV

Karena project ini adalah simulasi, kita membuat data dummy e-commerce dalam bentuk CSV.

File CSV diletakkan di folder:

```
seeds/
```

File yang dibuat:

```
raw_customers.csv
raw_products.csv
```

```
raw_orders.csv
raw_order_items.csv
raw_payments.csv
raw_refunds.csv
```

Penjelasan masing-masing file:

File	Isi
<code>raw_customers.csv</code>	Data customer
<code>raw_products.csv</code>	Data produk
<code>raw_orders.csv</code>	Data order
<code>raw_order_items.csv</code>	Detail item dalam order
<code>raw_payments.csv</code>	Data pembayaran
<code>raw_refunds.csv</code>	Data refund

Dalam dunia kerja nyata, data ini biasanya berasal dari aplikasi atau sistem bisnis. Dalam project ini, CSV digunakan sebagai pengganti data mentah tersebut.

9. Memasukkan CSV ke PostgreSQL dengan `dbt seed`

Command:

```
dbt seed
```

Tujuan:

Memasukkan file CSV dari folder seeds ke PostgreSQL sebagai tabel.

Contoh alur:

```
seeds/raw_orders.csv
↓
dbt seed
↓
public.raw_orders
```

Setelah `dbt seed`, tabel yang muncul di PostgreSQL:

```
public.raw_customers
public.raw_products
```

```
public.raw_orders
public.raw_order_items
public.raw_payments
public.raw_refunds
```

Tabel-tabel ini disebut sebagai **raw tables** karena masih berisi data mentah.

Karakteristik raw data:

```
belum dibersihkan
belum distandarkan
belum digabungkan
belum dihitung metrik bisnisnya
```

Command `dbt seed` digunakan saat:

```
ada file CSV baru
isi CSV berubah
ingin memuat ulang data seed ke database
```

10. Membuat Staging Layer

Setelah raw data masuk ke PostgreSQL, kita membuat layer pertama transformasi, yaitu **staging**.

Folder yang dibuat:

```
models/staging/
```

File yang dibuat:

```
stg_customers.sql
stg_products.sql
stg_orders.sql
stg_order_items.sql
stg_payments.sql
stg_refunds.sql
```

Tujuan staging:

```
membersihkan data mentah
menyamakan nama kolom
mengubah tipe data
```


menstandarkan nilai
membuang data tidak valid

Contoh pada `stg_orders.sql`:

```
select
  order_id,
  customer_id,
  cast(order_date as date) as order_date,

  case
    when lower(status) in ('completed', 'complete') then 'completed'
    when lower(status) in ('cancelled', 'canceled') then 'cancelled'
    when lower(status) = 'pending' then 'pending'
    else 'unknown'
  end as order_status

from {{ source('raw', 'raw_orders') }}
where order_id is not null
```

Sebelum staging, status order bisa berbeda-beda:

completed
Complete
COMPLETED

Setelah staging, semuanya distandarkan menjadi:

completed

Hasil staging:

stg_customers
stg_products
stg_orders
stg_order_items
stg_payments
stg_refunds

Staging model biasanya dibuat sebagai **view** karena hanya melakukan cleaning ringan.

11. Membuat Intermediate Layer

Setelah staging, kita membuat **intermediate layer**.

Folder:

```
models/intermediate/
```

File yang dibuat:

```
int_order_revenue.sql  
int_order_refunds.sql  
int_order_payments.sql  
int_customer_order_summary.sql
```

Tujuan intermediate:

```
membuat perhitungan sementara  
memecah query besar menjadi bagian kecil  
menyiapkan logic bisnis sebelum masuk ke model final  
membuat project lebih rapi dan mudah dirawat
```

11.1 int_order_revenue.sql

```
select  
    order_id,  
    sum(quantity * unit_price) as gross_revenue,  
    sum(quantity) as total_quantity,  
    count(order_item_id) as total_items  
from {{ ref('stg_order_items') }}  
group by order_id
```

Fungsi:

Menghitung gross revenue, total quantity, dan total item per order.

Contoh:

```
order 1001 punya beberapa item  
↓  
dijumlahkan menjadi gross_revenue per order
```

11.2 int_order_refunds.sql

```
select  
    order_id,
```

```

sum(refund_amount) as refund_amount,
count(refund_id) as total_refunds
from {{ ref('stg_refunds') }}
group by order_id

```

Fungsi:

Menghitung total refund per order.

11.3 int_order_payments.sql

```

select
  order_id,
  sum(payment_amount) as total_payment_amount,

  max(case
    when payment_status = 'paid' then 1
    else 0
  end) as has_paid_payment,

  max(payment_method) as payment_method

from {{ ref('stg_payments') }}
group by order_id

```

Fungsi:

Meringkas data pembayaran per order.

11.4 int_customer_order_summary.sql

```

select
  o.customer_id,
  count(distinct o.order_id) as total_orders,
  min(o.order_date) as first_order_date,
  max(o.order_date) as last_order_date
from {{ ref('stg_orders') }} o
where o.order_status = 'completed'
group by o.customer_id

```

Fungsi:

Meringkas jumlah order, order pertama, dan order terakhir setiap customer.

Intermediate layer biasanya belum dipakai langsung oleh dashboard. Ia hanya menjadi layer bantuan untuk membangun model final.

12. Membuat Marts Layer

Setelah intermediate, kita membuat layer final yaitu **marts**.

Folder:

```
models/marts/
```

File yang dibuat:

```
dim_customers.sql  
dim_products.sql  
fct_sales.sql  
mart_monthly_revenue.sql  
mart_product_performance.sql  
mart_customer_segments.sql
```

Tujuan marts:

membuat tabel final yang siap digunakan untuk dashboard dan analisis bisnis.

13. Perbedaan **dim**, **fct**, dan **mart**

Di layer marts, kita membuat beberapa jenis tabel.

13.1 Dimension Table: **dim_***

Dimension table berisi data deskriptif atau data master.

Contoh:

```
dim_customers  
dim_products
```

Fungsi **dim_customers**:

```
menyimpan informasi customer  
menyimpan kota customer  
menyimpan tanggal signup  
menyimpan segmentasi customer
```

Fungsi `dim_products`:

```
menyimpan nama produk  
menyimpan kategori produk  
menyimpan cost produk
```

Dimension table biasanya menjawab pertanyaan:

```
siapa?  
apa?  
kategori apa?  
di mana?
```

13.2 Fact Table: `fct_*`

Fact table berisi data transaksi dan angka utama.

Contoh:

```
fct_sales
```

Fungsi `fct_sales`:

```
menyimpan order  
menyimpan customer  
menyimpan tanggal transaksi  
menyimpan gross revenue  
menyimpan refund  
menyimpan net revenue  
menyimpan jumlah item  
menyimpan metode pembayaran
```

Fact table biasanya menjawab pertanyaan:

```
berapa?  
kapan terjadi?
```

transaksinya apa?
nilainya berapa?

13.3 Business Mart: `mart_*`

Business mart adalah tabel ringkasan untuk kebutuhan analisis tertentu.

Contoh:

`mart_monthly_revenue`
`mart_product_performance`
`mart_customer_segments`

Fungsi:

Model	Fungsi
<code>mart_monthly_revenue</code>	Ringkasan revenue bulanan
<code>mart_product_performance</code>	Ringkasan performa produk
<code>mart_customer_segments</code>	Ringkasan jumlah customer per segment

Business mart biasanya dipakai langsung untuk visualisasi seperti grafik bulanan, ranking produk, dan segmentasi customer.

14. Detail Model Marts

14.1 `dim_customers.sql`

```
select
  c.customer_id,
  c.customer_name,
  c.email,
  c.city,
  c.signup_date,

  coalesce(s.total_orders, 0) as total_orders,
  s.first_order_date,
  s.last_order_date,

  case
    when coalesce(s.total_orders, 0) >= 3 then 'loyal_customer'
    when coalesce(s.total_orders, 0) = 1 then 'new_customer'
    when coalesce(s.total_orders, 0) = 2 then 'repeat_customer'
    else 'no_order'
```

```

        end as customer_segment

from {{ ref('stg_customers') }} c
left join {{ ref('int_customer_order_summary') }} s
on c.customer_id = s.customer_id

```

Tujuan:

Membuat tabel profil customer yang sudah dilengkapi total order dan segmentasi.

14.2 dim_products.sql

```

select
    product_id,
    product_name,
    category,
    cost
from {{ ref('stg_products') }}

```

Tujuan:

Membuat tabel master produk.

14.3 fct_sales.sql

```

select
    o.order_id,
    o.customer_id,
    o.order_date,
    o.order_status,

    p.payment_method,
    p.total_payment_amount,
    p.has_paid_payment,

    r.gross_revenue,
    coalesce(f.refund_amount, 0) as refund_amount,
    r.gross_revenue - coalesce(f.refund_amount, 0) as net_revenue,

    r.total_quantity,
    r.total_items

```

```

from {{ ref('stg_orders') }} o
left join {{ ref('int_order_revenue') }} r
    on o.order_id = r.order_id
left join {{ ref('int_order_refunds') }} f
    on o.order_id = f.order_id
left join {{ ref('int_order_payments') }} p
    on o.order_id = p.order_id
where o.order_status = 'completed'

```

Tujuan:

Membuat fact table utama untuk analisis penjualan.

Metric penting:

```

gross_revenue = revenue sebelum refund
refund_amount = nilai refund
net_revenue = gross_revenue - refund_amount

```

14.4 mart_monthly_revenue.sql

```

select
    date_trunc('month', order_date)::date as revenue_month,
    count(distinct order_id) as total_orders,
    count(distinct customer_id) as total_customers,
    sum(gross_revenue) as gross_revenue,
    sum(refund_amount) as refund_amount,
    sum(net_revenue) as net_revenue,
    avg(net_revenue) as average_order_value
from {{ ref('fct_sales') }}
group by 1
order by 1

```

Tujuan:

Membuat ringkasan revenue bulanan.

14.5 mart_product_performance.sql

```

select
    p.product_id,
    p.product_name,

```



```

p.category,

sum(oi.quantity) as total_quantity_sold,
sum(oi.quantity * oi.unit_price) as gross_revenue,
sum(oi.quantity * (oi.unit_price - p.cost)) as estimated_gross_profit

from {{ ref('stg_order_items') }} oi
left join {{ ref('stg_orders') }} o
  on oi.order_id = o.order_id
left join {{ ref('dim_products') }} p
  on oi.product_id = p.product_id
where o.order_status = 'completed'
group by
  p.product_id,
  p.product_name,
  p.category
order by gross_revenue desc

```

Tujuan:

Membuat ringkasan performa produk berdasarkan quantity sold, gross revenue, dan estimasi profit.

14.6 mart_customer_segments.sql

```

select
  customer_segment,
  count(customer_id) as total_customers,
  sum(total_orders) as total_orders
from {{ ref('dim_customers') }}
group by customer_segment
order by total_customers desc

```

Tujuan:

Membuat ringkasan jumlah customer berdasarkan segmentasi.

15. Menjalankan Transformasi dengan dbt run

Setelah semua model dibuat, kita menjalankan:

```
dbt run
```

Tujuan:

```
Menjalankan semua file SQL di folder models dan membuat hasilnya di PostgreSQL.
```

Command ini membuat:

```
staging models  
intermediate models  
marts models
```

Hasil awalnya:

```
stg_* muncul sebagai view  
int_* muncul sebagai view  
dim_*, fct_*, mart_* muncul sebagai table
```

Setelah schema dipisahkan, hasilnya menjadi:

```
public_staging.stg_*  
public_intermediate.int_*  
public_marts.dim_*  
public_marts.fct_*  
public_marts.mart_*
```

Command `dbt run` digunakan saat:

```
baru membuat model SQL  
mengubah logic SQL  
ingin memperbarui hasil transformasi  
ingin membangun ulang tabel/view di database
```

16. Membuat Testing dan Dokumentasi Model

Kita membuat file:

```
models/schema.yml
```

Tujuan file ini:

memberi deskripsi model
memberi deskripsi kolom
menambahkan data test
membantu dbt docs menghasilkan dokumentasi yang lengkap

Contoh test:

```
- name: order_id
  tests:
    - not_null
    - unique
```

Artinya:

order_id tidak boleh kosong
order_id tidak boleh duplikat

Contoh accepted values:

```
- name: order_status
  tests:
    - accepted_values:
        values: ['completed', 'cancelled', 'pending', 'unknown']
```

Artinya:

order_status hanya boleh berisi completed, cancelled, pending, atau unknown.

Jenis test yang digunakan:

Test	Fungsi
<code>not_null</code>	Memastikan kolom tidak kosong
<code>unique</code>	Memastikan nilai tidak duplikat
<code>accepted_values</code>	Memastikan isi kolom hanya sesuai daftar nilai tertentu

17. Menjalankan Data Quality Test dengan `dbt test`

Command:

```
dbt test
```

Tujuan:

Mengecek apakah data sudah memenuhi aturan kualitas data yang ditentukan di `schema.yml`.

Contoh pengecekan:

```
customer_id tidak boleh null  
order_id tidak boleh duplikat  
order_status harus valid  
payment_status harus valid  
net_revenue tidak boleh null
```

Jika test berhasil, output menunjukkan status `PASS`.

Dalam project ini, sempat muncul error dari folder bawaan dbt:

```
models/example/schema.yml
```

Solusinya adalah menghapus folder:

```
models/example/
```

Karena folder tersebut hanya contoh bawaan dari `dbt init`, bukan bagian dari project e-commerce kita.

Setelah folder example dihapus, test project berhasil berjalan dengan baik.

Command `dbt test` digunakan saat:

```
setelah dbt run  
setelah mengubah schema.yml  
setelah menambahkan test baru  
sebelum data dipakai dashboard
```

18. Membuat Dokumentasi dan Lineage dengan `dbt docs`

Command pertama:

```
dbt docs generate
```

Tujuan:

Membuat dokumentasi otomatis dari model, kolom, test, dan hubungan antar-model.

Command kedua:

```
dbt docs serve
```

Tujuan:

Membuka dokumentasi dbt di browser lokal.

Di dbt docs, kita bisa melihat:

```
daftar model  
deskripsi tabel  
deskripsi kolom  
SQL model  
test yang digunakan  
lineage / DAG
```

Lineage menunjukkan alur data, misalnya:

```
raw_orders  
↓  
stg_orders  
↓  
fct_sales  
↓  
mart_monthly_revenue
```

Manfaat dbt docs:

```
memudahkan dokumentasi project  
memudahkan tracking asal data  
memudahkan debugging  
memudahkan memahami dampak perubahan model  
meningkatkan kualitas portfolio
```

19. Membuat Source Configuration

Awalnya, staging model mengambil raw data dengan:

```
from {{ ref('raw_orders') }}
```

Namun, secara konsep, `raw_orders` bukan model transformasi, melainkan data sumber.

Maka kita membuat file:

```
models/staging/sources.yml
```

Isi sumber data:

```
version: 2

sources:
  - name: raw
    database: ecommerce_analytics
    schema: public
    description: "Raw e-commerce data loaded from CSV seed files into PostgreSQL."

    tables:
      - name: raw_customers
        description: "Raw customer data containing customer identity, city, email, and signup date."

      - name: raw_products
        description: "Raw product data containing product name, category, and cost."

      - name: raw_orders
        description: "Raw order data containing order date, customer ID, and original order status."

      - name: raw_order_items
        description: "Raw order item data containing product quantity and unit price per order."

      - name: raw_payments
        description: "Raw payment data containing payment amount, payment method, and payment status."

      - name: raw_refunds
```

```
description: "Raw refund data containing refund amount, refund date,  
and refund reason."
```

Setelah itu, staging model diubah dari:

```
from {{ ref('raw_orders') }}
```

menjadi:

```
from {{ source('raw', 'raw_orders') }}
```

Tujuan penggunaan `source()`:

- membedakan raw data dengan model transformasi
- membuat lineage lebih jelas
- membuat project lebih profesional
- membuat source data terdokumentasi

Perbedaan:

Sebelum	Sesudah
<code>{{ ref('raw_orders') }}</code>	<code>{{ source('raw', 'raw_orders') }}</code>
Raw table dianggap seperti model	Raw table dianggap sebagai source
Lineage kurang rapi	Lineage lebih jelas
Cocok untuk latihan awal	Cocok untuk portfolio profesional

20. Mengatur Schema Output di `dbt_project.yml`

Awalnya semua hasil dbt masuk ke schema `public`.

Agar lebih rapi, kita mengatur folder model agar masuk ke schema berbeda.

File yang diubah:

```
dbt_project.yml
```

Konfigurasi:

```
models:  
  dbt_ecommerce:
```

```

staging:
  +schema: staging
  +materialized: view

intermediate:
  +schema: intermediate
  +materialized: view

marts:
  +schema: marts
  +materialized: table

```

Tujuannya:

memisahkan hasil transformasi berdasarkan layer
 membuat database lebih rapi
 membuat workflow mirip dunia kerja profesional

Hasil akhirnya:

Folder dbt	Schema PostgreSQL	Materialization
models/staging	public_staging	View
models/intermediate	public_intermediate	View
models/marts	public_marts	Table

Kenapa menjadi `public_staging`, bukan hanya `staging`?

Karena di `profiles.yml`, schema default adalah:

```
schema: public
```

Lalu dbt menambahkan nama custom schema di belakangnya.

Hasil akhir di PostgreSQL:

```

public
├─ raw_customers
├─ raw_products
├─ raw_orders
├─ raw_order_items
├─ raw_payments
└─ raw_refunds

public_staging
├─ stg_customers

```



```
|— stg_products
|— stg_orders
|— stg_order_items
|— stg_payments
└─ stg_refunds

public_intermediate
|— int_order_revenue
|— int_order_refunds
|— int_order_payments
└─ int_customer_order_summary

public_marts
|— dim_customers
|— dim_products
|— fct_sales
|— mart_monthly_revenue
|— mart_product_performance
└─ mart_customer_segments
```

21. Perbedaan Command dbt yang Digunakan

21.1 dbt debug

```
dbt debug
```

Fungsi:

```
Mengecek koneksi dan konfigurasi project.
```

Dipakai saat:

```
setup awal
mengubah profiles.yml
database error
koneksi gagal
```

21.2 dbt seed

```
dbt seed
```

Fungsi:

Memasukkan file CSV dari folder seeds ke database sebagai tabel.

Dipakai saat:

ada file CSV baru
isi CSV berubah
ingin memuat ulang raw dummy data

21.3 dbt run

dbt run

Fungsi:

Menjalankan transformasi SQL di folder models.

Dipakai saat:

membuat model baru
mengubah file SQL
ingin memperbarui staging, intermediate, atau marts

21.4 dbt test

dbt test

Fungsi:

Mengecek kualitas data berdasarkan aturan di file YAML.

Dipakai saat:

setelah dbt run
setelah menambah test
sebelum data dipakai dashboard

21.5 dbt docs generate

```
dbt docs generate
```

Fungsi:

```
Membuat dokumentasi dbt.
```

21.6 dbt docs serve

```
dbt docs serve
```

Fungsi:

```
Membuka dokumentasi dbt di browser lokal.
```

22. Urutan Command yang Digunakan dalam Project

Urutan awal:

```
dbt debug
```

Lalu memasukkan CSV:

```
dbt seed
```

Lalu menjalankan transformasi:

```
dbt run
```

Lalu mengecek kualitas data:

```
dbt test
```

Lalu membuat dokumentasi:

```
dbt docs generate
dbt docs serve
```

Jika ada perubahan besar pada project, urutan aman yang bisa digunakan:

```
dbt debug
dbt seed
dbt run
dbt test
dbt docs generate
```

Namun dalam praktik harian:

```
ubah SQL model → dbt run → dbt test
ubah CSV seed → dbt seed → dbt run → dbt test
ubah koneksi → dbt debug
ubah dokumentasi → dbt docs generate
```

23. Alur Akhir Project Sebelum Dashboard

Sampai tahap ini, alur project sudah menjadi:

```
CSV dummy
↓
dbt seed
↓
public.raw_*
↓
source()
↓
public_staging.stg_*
↓
ref()
↓
public_intermediate.int_*
↓
ref()
↓
public_marts.dim_*
public_marts.fct_*
public_marts.mart_*
↓
dbt test
```

↓
dbt docs

Secara bisnis, alurnya:

Data mentah e-commerce
↓
dibersihkan
↓
dihitung metrik awal
↓
dibentuk menjadi tabel final
↓
diuji kualitasnya
↓
didokumentasikan
↓
siap dipakai dashboard

24. Hasil Akhir yang Sudah Dibangun

Raw Tables

```
public.raw_customers  
public.raw_products  
public.raw_orders  
public.raw_order_items  
public.raw_payments  
public.raw_refunds
```

Staging Views

```
public_staging.stg_customers  
public_staging.stg_products  
public_staging.stg_orders  
public_staging.stg_order_items  
public_staging.stg_payments  
public_staging.stg_refunds
```

Intermediate Views

```
public_intermediate.int_order_revenue  
public_intermediate.int_order_refunds
```

```
public_intermediate.int_order_payments
public_intermediate.int_customer_order_summary
```

Marts Tables

```
public_marts.dim_customers
public_marts.dim_products
public_marts.fct_sales
public_marts.mart_monthly_revenue
public_marts.mart_product_performance
public_marts.mart_customer_segments
```

25. Tujuan Besar dari Semua Tahap Ini

Semua tahap yang sudah dilakukan bertujuan untuk membangun workflow data yang profesional.

Bukan hanya:

```
Data mentah → Power BI
```

Tetapi:

```
Data mentah
↓
Data cleaning
↓
Data modeling
↓
Data quality testing
↓
Data documentation
↓
Dashboard-ready tables
```

Dengan project ini, kita menunjukkan skill:

```
SQL transformation
dbt modeling
PostgreSQL
data warehouse layering
data quality testing
data documentation
```

data lineage
analytics engineering workflow

26. Kesimpulan Sebelum Dashboard

Sebelum masuk ke Power BI, project ini sudah berhasil membangun pipeline analytics sederhana menggunakan dbt dan PostgreSQL.

Data mentah dari CSV berhasil dimasukkan ke PostgreSQL menggunakan `dbt seed`. Kemudian data tersebut dibersihkan di staging layer, dihitung di intermediate layer, dan dibentuk menjadi tabel final di marts layer. Setelah itu, kualitas data diuji menggunakan `dbt test`, dokumentasi dibuat menggunakan `dbt docs`, dan struktur schema dirapikan agar menyerupai workflow profesional.

Dengan demikian, data di schema `public_marts` sudah siap digunakan oleh Power BI untuk membuat dashboard bisnis.